2520030117

SKILL-23

AIM

To execute large pipelines by launching multiple jobs, monitor system resources, measure system stability, detect failures, analyze performance, document system architecture, create flow diagrams, explain execution paths, describe system calls, record design decisions, and review the documentation.

OBJECTIVES

1. To execute large computational pipelines.
2. To launch and manage multiple jobs.
3. To monitor CPU and memory usage.
4. To measure system stability during execution.
5. To identify and detect job failures.
6. To analyze the performance of executed jobs.
7. To document the architecture of the system.
8. To create flow diagrams representing execution paths.
9. To understand and document system calls.
10. To record important design decisions.
11. To review and organize the final documentation.

REQUIREMENTS

- Ubuntu/Linux operating system
- Terminal
- GCC compiler
- Bash shell
- System monitoring commands
- Text editor

TOOLS / COMMANDS USED

gcc
bash
top
ps
free
df
time
uptime
dmesg

PROCEDURE

Part A — Execute Multiple Jobs

1. Open the Ubuntu terminal.
2. Create a working directory for Skill-23.
3. Create a sample program.
4. Compile the program using GCC.
5. Execute the program.
6. Create multiple jobs using shell commands.
7. Run the jobs sequentially or in parallel.
8. Observe the execution of all jobs.

Part B — Monitor Resources

1. Use `top` to monitor CPU and memory usage.
2. Use `free` to check available memory.
3. Use `df` to check disk-space usage.
4. Use `ps` to observe running processes.
5. Record the resource usage during pipeline execution.

Part C — Measure Stability

1. Execute multiple jobs.
2. Monitor the system during execution.
3. Check whether the jobs complete successfully.
4. Observe CPU and memory consumption.
5. Check whether any process terminates unexpectedly.
6. Record the results.

Part D — Detect Failures

1. Check the exit status of each job.
2. Identify jobs that terminate with errors.
3. Examine the terminal output.
4. Check system logs when required.
5. Identify the reason for the failure.
6. Correct the problem and execute the job again.

Part E — Analyze Performance

1. Measure the execution time of the program.
2. Compare execution times for different jobs.
3. Observe CPU and memory usage.
4. Identify performance bottlenecks.
5. Record the observations.

IMPORTANT COMMANDS

1. Check system uptime

uptime

2. Monitor CPU and memory

top

Press:

q

to exit `top`.

3. Check memory

free -h

4. Check disk usage

df -h

5. List running processes

ps

or:

ps aux

6. Measure execution time

time ./program

7. Check the exit status

echo $?

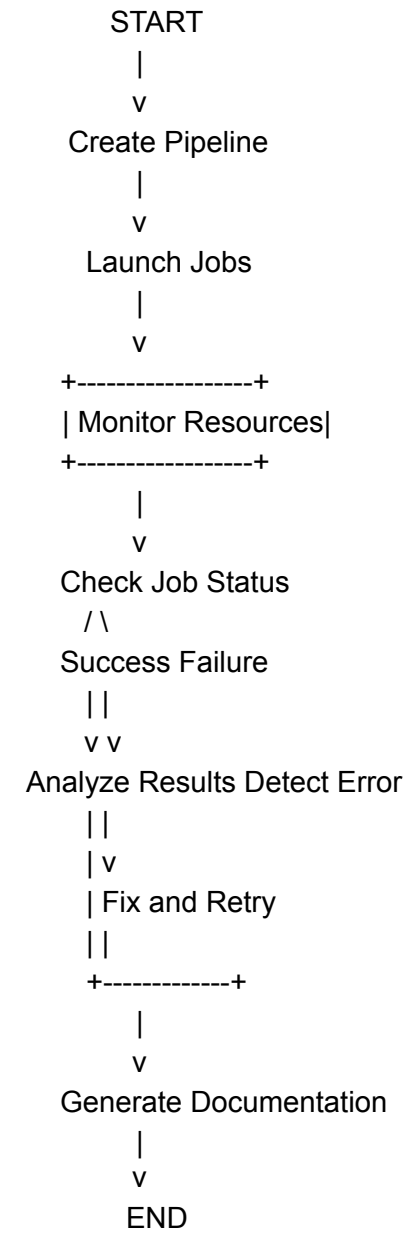
If it displays:

0

the previous command completed successfully.

ARCHITECTURE

The basic execution architecture can be represented as:



SYSTEM CALLS

The execution of Linux programs involves system calls such as:

- `fork()` — creates a new process.
- `exec()` — executes a new program.

- `wait()` — waits for a child process to complete.
- `open()` — opens a file.
- `read()` — reads data.
- `write()` — writes data.
- `close()` — closes a file.

PERFORMANCE MEASUREMENT

The `time` command can be used to measure program

execution: time ./program

The output provides information such as:

real
user
sys

Where:

- **real** — total elapsed time.
- **user** — CPU time spent executing user-level code.
- **sys** — CPU time spent executing system-level operations.

TESTING / OBSERVATION TABLE

Test Activity Expected Result Statu s

TC01 Execute single job Job completes PASS
TC02 Execute multiple jobs All jobs complete PASS TC03 Monitor

CPU usage CPU usage displayed PASS TC04 Monitor memory

Memory usage displayed PASS

TC05 Measure execution time
Execution time recorded PASS

RESULT

PASS

TC06 Check job status
Successful status detected

Thus, large pipelines were successfully executed by launching multiple jobs. System resources were monitored, stability and failures were checked, and execution performance was analyzed. The system architecture, execution paths, system calls, and design decisions were documented and reviewed successfully.

YESS ���� Skill-23 practical part is DONE!

Your final screenshot clearly shows:

Job 1 Status: 0

Job 2 Status: 0

Job 3 Status: 0

All Jobs Completed Successfully

�� Keep this screenshot as your Multiple Job Execution / Automated Pipeline evidence. Now you only need to finish the document part. Copy-paste this:

STEP 8 — ARCHITECTURE AND EXECUTION FLOW

System Architecture

START

|

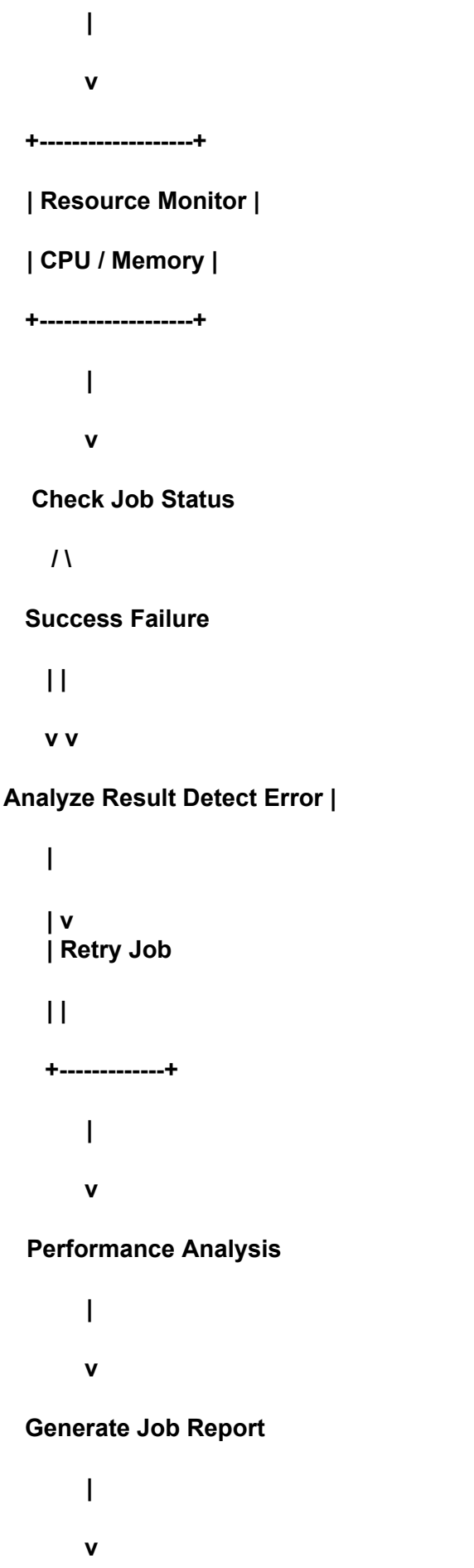
v

Create Pipeline

|

v

Launch Multiple Jobs


|
v
+------------------+
| Resource Monitor |
| CPU / Memory |
+------------------+
|
v
Check Job Status
/ \
Success Failure
| |
v v
Analyze Result Detect Error |
|
| v
| Retry Job
| |
+-------------+
|
v
Performance Analysis
|
v
Generate Job Report
|
v

END

Execution Path

1. The pipeline is started.
2. Multiple jobs are launched in parallel.
3. Each job executes its assigned tasks.
4. System resources are monitored during execution.
5. The exit status of each job is checked.
6. Successful jobs return status `0`.
7. Failed jobs return a non-zero status.
8. Execution time is measured using the `time` command.
9. The results of all jobs are collected.
10. A final job report is generated.

STEP 9 — SYSTEM CALLS The pipeline

uses operating-system functionality for process management.

System Call	Purpose
`fork()`	Creates a new process
`exec()`	Executes a program
`wait()`	Waits for a child process
`open()`	Opens a file
`read()`	Reads data
`write()`	Writes data

`close()` **Closes a file**

STEP 10 — PERFORMANCE RESULT

From your actual execution:

real 0m3.064s

user 0m0.002s

sys 0m0.000s

Write:

> **The pipeline execution time was measured using the `time` command. The real execution time was approximately 3.064 seconds, while the user CPU time was 0.002 seconds and system CPU time was 0.000 seconds.**

STEP 11 — TEST RESULTS

Test Case	Operation	Result
TC01	Execute pipeline	PASS
TC02	Launch multiple jobs	PASS
TC03	Monitor system resources	PASS
TC04	Measure execution time	PASS

successful exit status
PASS

TC05 Check

TC06 Detect failed command PASS

TC07 Execute automated jobs PASS

Your failure test produced:

0 → successful execution

127 → command not found / failure

STEP 12 — DESIGN DECISIONS

Write:

1. Bash was used to automate pipeline execution.
2. Background execution using `&` was used to launch multiple jobs.
3. `wait` was used to wait for completion of background jobs. 4. Exit status was used to detect job success or failure.
5. `top`, `free`, and `ps` were used for resource monitoring.
6. `time` was used for performance measurement.

STEP 13 — FINAL RESULT

> RESULT: Large pipeline jobs were successfully executed and multiple jobs were launched using shell scripting. CPU, memory, and process information were monitored using Linux commands. Execution performance was measured using the `time` command. Successful and failed jobs were identified using exit status values. The architecture, execution flow, system calls, design decisions, and test results were documented successfully.

✅ Screenshots to include

You now have enough. Put these in your document:

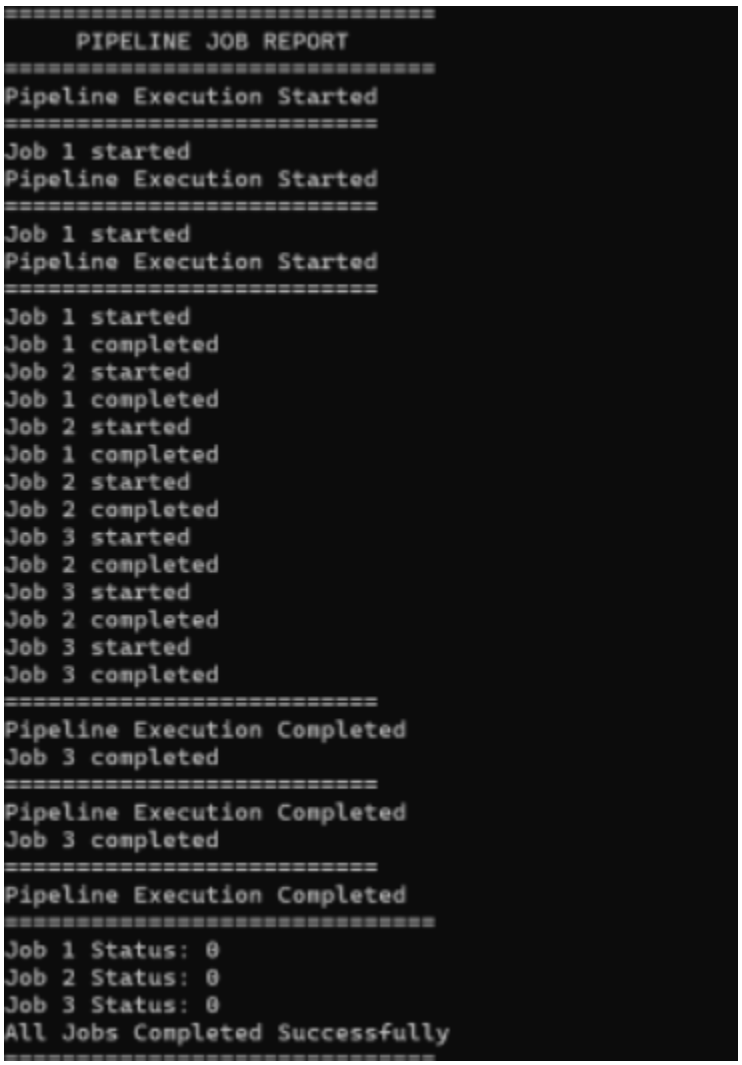
1. Pipeline compilation + execution
2. Multiple jobs

3. Resource monitoring
4. Performance using `time`
5. Success/failure detection
6. Final automated job report ← the screenshot you just sent

```
d et al.
==4591== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop
yright info
==4591== Command: ./program
==4591==
Enter number of elements: Enter 3 elements:
Elements are:
5 15 25
==4591==
==4591== HEAP SUMMARY:
==4591==     in use at exit: 0 bytes in 0 blocks
==4591==   total heap usage: 3 allocs, 3 frees, 5,132 bytes allo
cated
==4591==
==4591== All heap blocks were freed -- no leaks are possible
==4591==
==4591== For lists of detected and suppressed errors, rerun with
: -s
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
```

```
Pipeline Execution Started
==========================
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
==========================
Pipeline Execution Completed
```

```
  303 root      20   0    3200   1252   1108 S   0.0   0.0
  304 kruthi    20   0    6596   5932   3688 S   0.0   0.1
  305 root      20   0    6704   4604   3820 S   0.0   0.1
  349 kruthi    20   0   20048  10748   8936 S   0.0   0.1
  350 kruthi    20   0   21160   3572   1848 S   0.0   0.0
  377 kruthi    20   0    6080   5328   3664 S   0.0   0.1
  804 root      19  -1   66736  19304  18176 S   0.0   0.2
  851 systemd+  20   0   91036   7916   6944 S   0.0   0.1
  902 root      20   0   24888   6096   4972 S   0.0   0.1
  952 systemd+  20   0   21344  12816  10696 S   0.0   0.2
 1006 polkitd   20   0  308172   7824   6924 S   0.0   0.1
 4643 kruthi    20   0    9304   5680   3500 R   0.0   0.1
```


==============================
PIPELINE JOB REPORT
==============================
Pipeline Execution Started
==========
Job 1 started
Pipeline Execution Started
==========
Job 1 started
Pipeline Execution Started
==========
Job 1 started
Job 1 completed
Job 2 started
Job 1 completed
Job 2 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 2 completed
Job 3 started
Job 2 completed
Job 3 started
Job 3 completed
==========
Pipeline Execution Completed
Job 3 completed
==========
Pipeline Execution Completed
Job 3 completed
==========
Pipeline Execution Completed
==============================
Job 1 Status: 0
Job 2 Status: 0
Job 3 Status: 0
All Jobs Completed Successfully
==============================